Digital Signal Processing Exercise Sheet 4

Sampling

Script: Chapter 4 (Sampling and Quantisation) **with solutions**

Learning goals

- Derive and apply the quantisation-noise model and the rule SNR $\approx 6.02N + 1.76$ dB; size an ADC for the knock sensor.
- Configure timer-driven sampling on a microcontroller and verify the sampling rate and the conversion time with the logic analyzer.
- Quantify the effect of sampling-time jitter and compare it with quantisation noise.
- Resample a time-sampled engine signal onto a crank-angle grid using the edges of the 60-2 crank signal, and explain when the time domain and when the angle domain is the natural one.

Setup: JSLinux

From this sheet on, the lab exercises also run in JSLinux, a Linux system in the browser. If you have not done the JSLinux bonus exercises of Sheets 0–2, set it up now (about 10 min):

1. Open `https://bellard.org/jslinux`, start the *x86 Alpine Linux* (console) virtual machine and check the compiler with `gcc -version` (or `tcc -v`).
2. Upload the course headers `engine_signals.h`, `asciiplot.h`, `csvio.h` (`code/common/`) and the files of the exercise (e.g. `code/jslinux/sheet04/`) with the upload button below the terminal; check with `ls` that they arrived. Headers and program must be in the same directory.
3. Compile and run, e.g.

```
gcc -O2 -o quant quant.c -lm
./quant
```

4. The VM forgets all files when the page is reloaded: keep your sources on your own computer and upload them again. Text from Wokwi (Serial Monitor) gets into the VM with `cat > data.csv`, paste, `Ctrl-D`; `export_file data.csv` downloads a file from the VM to your computer.

Background

An ADC with N bits and full-scale range V_{FS} has the step size (1 LSB) $\Delta = V_{\mathrm{FS}}/2^N$. If the signal is "busy" (it moves over many steps between samples), the quantisation error $e = x_q - x$ is well modelled as white noise, uniformly distributed in $[-\Delta/2, \Delta/2)$. Sampling instants that deviate randomly from the ideal grid, $t_n = nT_s + \delta_n$ with $\mathrm{std}(\delta_n) = \sigma_j$ (*jitter*), produce an additional error $e_n \approx \dot{x}(nT_s)\,\delta_n$ that grows with the signal frequency.

An ECU samples two kinds of signals: *time-based* (fixed T_s, e.g. the knock sensor with 20 kHz to 50 kHz) and *angle-based* (one value per fixed crank-angle increment, triggered by the crank wheel). An angle grid can also be computed afterwards from time samples: the crank edges provide pairs (time, angle), between which the angle is interpolated.

Exercise 4.1 Quantisation and jitter budget

PAPER

a) Assume a uniformly distributed quantisation error. Derive its variance and show that a full-scale sine quantised with N bits has SNR $= 6.02N + 1.76$ dB. By how much does the SNR drop if the sine uses only 1/10 of the full-scale amplitude?

b) Compute 1 LSB and the rms quantisation noise for the Arduino Uno (10 bit, 0 V to 5 V) and the ESP32 (12 bit, 0 V to 3.3 V).

c) The conditioned knock-sensor signal spans 0 V to 5 V (offset 2.5 V) for the heaviest knock. The weakest knock that still has to be measured has an amplitude of 10 mV; its quantisation SNR must be at least 30 dB. Which dynamic range (in dB) is required, and how many bits does the ADC need?

d) The knock detector only uses a band of width $B = 1\,\text{kHz}$ around the knock frequency, the ADC samples at $f_s = 50\,\text{kHz}$. Assuming white quantisation noise, by how many dB does a band-pass filter improve the SNR of (c)? How many ADC bits does this "save"?

e) Show that jitter limits the SNR of a sine of frequency f to $\text{SNR}_j = -20\log_{10}(2\pi f\sigma_j)$. (i) Which jitter is allowed for a 6.5 kHz knock signal if jitter shall not degrade a 12-bit ADC? (ii) A sampling loop that polls `micros()` (resolution 4 µs on the Uno) produces a timing error uniformly distributed over 4 µs. Compute σ_j, SNR_j at 6.5 kHz and the *effective number of bits* $\text{ENOB} = (\text{SNR} - 1.76)/6.02$.

Solution

a) For $e \sim \mathcal{U}[-\Delta/2, \Delta/2)$: $\sigma_e^2 = \frac{1}{\Delta}\int_{-\Delta/2}^{\Delta/2} e^2\,\mathrm{d}e = \frac{\Delta^2}{12}$. A full-scale sine has amplitude $A = 2^{N-1}\Delta$ and power $P_s = A^2/2 = 2^{2N}\Delta^2/8$. Hence

$$\text{SNR} = \frac{P_s}{\sigma_e^2} = \frac{3}{2}\,2^{2N} \;\Rightarrow\; 10\log_{10}\text{SNR} = 10\log_{10}1.5 + 20N\log_{10}2 = 1.76 + 6.02N\ \text{dB}.$$

The noise power does not depend on the signal, so an amplitude of 1/10 of full scale lowers the SNR by $20\log_{10}10 = 20\,\text{dB}$ (equivalent to losing 3.3 bits). Every unused part of the input range costs resolution.
b) Uno: $\Delta = 5/1024\,\text{V} = 4.88\,\text{mV}$, $\sigma_e = \Delta/\sqrt{12} = 1.41\,\text{mV}$. ESP32: $\Delta = 3.3/4096\,\text{V} = 0.806\,\text{mV}$, $\sigma_e = 0.233\,\text{mV}$.
c) Dynamic range: $20\log_{10}(2.5\,\text{V}/10\,\text{mV}) = 48\,\text{dB}$. A sine of amplitude a has $\text{SNR} = (a^2/2)/(\Delta^2/12) = 6a^2/\Delta^2$. $\text{SNR} \geq 10^3$ requires $\Delta \leq a\sqrt{6\cdot10^{-3}} = 0.775\,\text{mV}$, i.e. $2^N \geq 5\,\text{V}/0.775\,\text{mV} = 6455$, so $N = 13$ bits (a 12-bit ADC gives $\Delta = 1.22\,\text{mV}$ and only 26 dB). Equivalently: $48 + 30 = 78\,\text{dB}$ of "full-scale SNR" $\Rightarrow N \geq (78 - 1.76)/6.02 = 12.7$.
d) White quantisation noise is spread evenly over $[0, f_s/2]$; a filter of bandwidth B keeps the fraction $2B/f_s = 1/25$. The SNR improves by $10\log_{10}(f_s/2B) = 14\,\text{dB}$, i.e. $14/6.02 \approx 2.3$ bits: with the band-pass a 12-bit ADC reaches $26 + 14 = 40\,\text{dB}$ and even 11 bits ($\approx 34\,\text{dB}$) would suffice. (Oversampling and filtering trade sampling rate for resolution: 3 dB, i.e. half a bit, per doubling of f_s.)
e) For $x = A\sin\omega t$: $e_n \approx A\omega\cos(\omega n T_s)\,\delta_n$ with independent δ_n, so $\overline{e^2} = A^2\omega^2\sigma_j^2/2$ and $\text{SNR}_j = (A^2/2)/\overline{e^2} = 1/(\omega\sigma_j)^2$, i.e. $\text{SNR}_j = -20\log_{10}(2\pi f\sigma_j)$. (i) 12 bit: $\text{SNR}_q = 74.0\,\text{dB}$; $2\pi f\sigma_j \leq 10^{-74/20} = 2.0\cdot10^{-4}$ gives $\sigma_j \leq 4.9\,\text{ns}$ at 6.5 kHz – only hardware-triggered conversions achieve this. (ii) Uniform over 4 µs: $\sigma_j = 4\,\mu\text{s}/\sqrt{12} = 1.15\,\mu\text{s}$, $\text{SNR}_j = -20\log_{10}(2\pi\cdot6500\cdot1.15\cdot10^{-6}) = 26.5\,\text{dB}$, $\text{ENOB} = 4.1$ bits. A software-polled sampling loop turns a 10-bit ADC into a 4-bit ADC for knock signals!

Exercise 4.2 ADC in practice: timer-driven sampling WOKWI

Open the Uno project `code/wokwi/sheet04_adc/` (sketch, diagram and the `engine-sim` chip files). Timer1 triggers an interrupt service routine (ISR) at $f_s = 5\,\text{kHz}$ that samples the `KNOCK` output of the chip (A1) into a RAM buffer of 400 samples; the buffer is printed afterwards (the UART cannot transmit 5 kHz × a CSV line in real time). Pin 8 toggles at every sample (logic analyzer D0), pin 9 is HIGH while the ISR runs (D1), the chip's `TDC` output (pin 2, INT0) marks combustion TDCs.

a) Timer1 runs in CTC mode with prescaler 8. Compute the compare value `OCR1A` for 5 kHz and

complete `timer1_start()`. What is the lowest sampling rate possible with this prescaler?

b) Complete the ISR. Record about 0.1 s with the logic analyzer and determine from the VCD file (i) the sampling period (D0) and (ii) the time the ISR needs (pulse width on D1). Which sampling rate is the theoretical maximum for this structure? Why is the practical limit lower?

c) Between combustions, 140° to 175° after each TDC, the knock signal contains only sensor noise. Complete the computation of the index range of this quiet window (at 3000 rpm) and run the sketch with the noise slider at 50 mV, 10 mV and 0 mV. Compare the printed standard deviation (in LSB) with your expectation (chip output = 2.5 V + 0.6· sensor signal). Would the 12-bit ESP32 ADC improve the knock measurement here?

d) Turn the potentiometer to an arbitrary position and read the code several times. Is the quantisation error of this constant input "noise"? What does this mean for the validity of the model of Exercise 4.1?

Wokwi measures time in *simulated* time; the Uno core is simulated instruction by instruction, but treat the measured durations as indicative.

Solution

a) Timer clock $f_T = 16\,\text{MHz}/8 = 2\,\text{MHz}$; the period is $(\texttt{OCR1A}+1)/f_T$, so $\texttt{OCR1A} = 2\cdot10^6/5000 - 1 = 399$. The 16-bit register allows at most $65536/f_T = 32.8\,\text{ms}$, i.e. $f_{s,\min} = 30.5\,\text{Hz}$ (prescaler 64/256/1024 for slower rates).

```
/* ---------------- Timer1 compare-match interrupt: one sample ---------- */
ISR(TIMER1_COMPA_vect)
{
    PORTB |= _BV(PB1);                  /* pin 9 HIGH: ISR busy            */
    PINB = _BV(PB0);                    /* writing 1 to PINx toggles pin 8 */
    uint16_t v = analogRead(A1);        /* ~112 us on the Uno              */
    if (tdc_seen) { v |= 0x8000; tdc_seen = 0; }
    buf[nbuf++] = v;
    if (nbuf >= NBUF) { TIMSK1 = 0; done = true; }   /* block complete     */
    PORTB &= ~_BV(PB1);                 /* pin 9 LOW                       */
}

/* ---------------- (a) Timer1: CTC mode, prescaler 8 ------------------- */
void timer1_start()
{
    noInterrupts();
    TCCR1A = 0;                         /* no PWM outputs                  */
    TCCR1B = _BV(WGM12) | _BV(CS11);    /* CTC (TOP = OCR1A), clk/8 = 2 MHz */
    OCR1A  = F_CPU / 8 / FS_HZ - 1;     /* 16e6/8/5000 - 1 = 399           */
    TCNT1  = 0;
    TIFR1  = _BV(OCF1A);                /* clear a pending flag            */
    TIMSK1 = _BV(OCIE1A);               /* enable compare-match interrupt  */
    interrupts();
}
```

b) D0 changes level every 200 µs (square wave with period 400 µs). The D1 pulses last about 110 µs: the conversion itself takes 13 ADC clock cycles at 16 MHz/128 = 125 kHz, i.e. 104 µs, plus the overhead of `analogRead()` and the ISR entry/exit. Theoretical maximum: $1/112\,\mu\text{s} \approx 8.9\,\text{kHz}$ – then the CPU does nothing but sampling. In practice other interrupts (Timer0 for `millis()`, INT0, UART) must fit in between and the main program needs CPU time, so about 5 kHz to 7 kHz is a sensible limit with `analogRead()`. Single edges on D0 may be delayed by a few µs when the Timer0 ISR is running at the compare match: this is jitter (it can be removed by disabling Timer0, see Sheet 5).

c) Samples per degree: $f_s/(6\,n) = 5000/18000 = 0.278$, so $i_0 = \lceil 140 \cdot 0.278 \rceil = 39$, $i_1 = \lfloor 175 \cdot 0.278 \rfloor = 48$ (10 samples per TDC, about 70 per block). Expected standard deviation:

noise slider	sensor noise at ADC	expected std	simulation (3990 samples)
50 mV	30 mV	6.15 LSB	6.06 LSB
10 mV	6 mV	1.26 LSB	1.25 LSB
0 mV	0	0	0.46 LSB

("expected" = $\sqrt{(0.6\sigma_{\text{noise}}/\Delta)^2 + 1/12}$; the simulation column was obtained with the same engine model sampled exactly like the sketch. A single block contains only ≈ 70 samples, so the printed value scatters by about $\pm 9\,\%$.) The quantisation noise (0.29 LSB) is negligible against the sensor noise ($\sqrt{6.14^2 + 0.29^2} = 6.15$, i.e. 0.01 dB worse): the measurement is limited by the *sensor*, not by the ADC; a 12-bit ADC would not help. Only for the noise-free signal does quantisation dominate: the offset 2.5 V lies exactly on the transition between codes 511 and 512, so tiny residues of the decaying bursts make the LSB flicker (std ≈ 0.5 LSB in the simulation; in Wokwi the result depends on how the simulator rounds and may be exactly 0).
d) The code is constant, e.g. 3.30 V gives $\lfloor 3.30 \cdot 1024/5 \rfloor = 675$ every time. The error is a fixed, signal dependent offset (up to 1 LSB), not a random variable: the noise model of 4.1 requires a signal that changes over many LSB between samples (or added noise/dither). For slowly varying signals the quantisation error is correlated and appears as distortion, not as white noise.

Exercise 4.3 Quantisation noise and jitter

JSLINUX

`code/jslinux/sheet04/quant.c` generates a sine of $f_0 = 1234.5$ Hz at $f_s = 50$ kHz (10^5 samples; f_0/f_s is not a simple fraction, so the error is not periodic). The full-scale range is $[-1, 1)$.

a) Implement the function `quantise()`, a mid-tread quantiser with N bits: rounding to the nearest step and clipping to the codes $q = -2^{N-1}, \ldots, 2^{N-1} - 1$. Implement `snr_db()`. Measure the SNR for $N = 4 \ldots 16$ at full scale and compare with $6.02N + 1.76$ dB. For which N does the formula fail, and why?

b) Repeat with an amplitude of 0.1 (−20 dBFS).

c) Implement jittered sampling (Gaussian δ_n, no quantisation) and measure the SNR for $f = 100\,\text{Hz} \ldots 20\,\text{kHz}$ and $\sigma_j = 0.1, 1, 4\,\mu\text{s}$. Compare with $-20\log_{10}(2\pi f \sigma_j)$.

d) Combine a 12-bit quantiser with jitter: compute the ENOB for the knock frequency 6.5 kHz. At which signal frequency are jitter noise and quantisation noise equal for $\sigma_j = 1\,\mu\text{s}$?

Solution

```c
static double quantise(double x, int nbits)
{
    double delta = 2.0 / (double)(1L << nbits);
    long q = (long)floor(x / delta + 0.5);
    long qmax = (1L << (nbits - 1)) - 1, qmin = -(1L << (nbits - 1));
    if (q > qmax) q = qmax;
    if (q < qmin) q = qmin;
    return q * delta;
}

/* SNR in dB of x (reference) and its distorted version y */
static double snr_db(const double *x, const double *y, int n)
{
    double ps = 0.0, pe = 0.0;
    int i;
    for (i = 0; i < n; i++) {
        ps += x[i] * x[i];
        pe += (y[i] - x[i]) * (y[i] - x[i]);
    }
    return 10.0 * log10(ps / pe);
}
```

Jittered sample: `y[n] = A*sin(2*M_PI*f*(t + sj*grand()) + 0.3);`
a/b) Program output (SNR in dB):

N	full scale	$6.02N + 1.76$	-20 dBFS	expected
4	24.74	25.84	5.27	5.84
5	31.20	31.86	10.56	11.86
6	37.47	37.88	18.80	17.88
8	49.75	49.92	29.78	29.92
10	61.89	61.96	42.18	41.96
12	73.97	74.00	53.97	54.00
14	86.03	86.04	66.10	66.04
16	98.10	98.08	78.08	78.08

From 8 bits on, the formula holds within 0.2 dB. For $N \leq 5$ the measured SNR is up to 1.1 dB lower: with few levels the error is no longer uniform and independent of the signal (it is a deterministic, signal-dependent staircase error with harmonics), and the amplitude $1 - \Delta/2$ needed to avoid clipping reduces the signal power. The -20 dBFS column is 20 dB lower (within ± 0.4 dB) for $N \geq 7$; for small N the reduced sine spans only a few codes and the deviations are larger (± 1.3 dB).
c) Measured vs. theory (dB):

f [Hz]	$\sigma_j = 0.1$ µs		1 µs		4 µs	
	meas.	theory	meas.	theory	meas.	theory
100	84.03	84.04	64.07	64.04	52.02	52.00
1000	64.04	64.04	44.03	44.04	31.99	32.00
6500	47.76	47.78	27.79	27.78	15.75	15.74
10500	43.65	43.61	23.63	23.61	11.60	11.57
20000	38.03	38.02	18.01	18.02	6.20	5.97

The SNR drops by 20 dB per decade of frequency and per decade of jitter. Only at 20 kHz with 4 µs ($2\pi f \sigma_j = 0.5$) does the first-order approximation $e \approx \dot{x}\,\delta$ become inaccurate.
d) 12 bit + jitter at 6.5 kHz: $\sigma_j = 0.1$ µs: 47.8 dB (ENOB 7.6); 1 µs: 27.8 dB (ENOB 4.3); 4 µs: 15.8 dB (ENOB 2.3). At 100 Hz and 0.1 µs the full 12 bits remain (ENOB 11.97). Equality of the two noise sources: $2\pi f \sigma_j = 10^{-74/20} \Rightarrow f = 31.8$ Hz for $\sigma_j = 1$ µs. Conclusion: for kHz signals the sampling clock, not the number of bits, decides the effective resolution; ECUs therefore start knock conversions by hardware timers or sample continuously with a fixed ADC clock.

Exercise 4.4 Time domain vs. crank-angle domain JSLINUX

In `code/jslinux/sheet04/angle_resample.c` the virtual engine accelerates from 1500 rpm to 4500 rpm within 1 s (`e.rpm` is changed during the simulation). The ion-current signal is sampled at $f_s = 50$ kHz; the rising crank edges are time-stamped with a resolution of 2 µs ("input capture" as in a real ECU), and the cam level is latched at every edge.

a) Assign a crank angle to every edge: detect the gap (edge interval $> 2\times$ the previous one), give the first edge after the first gap the angle 0° or 360° (cam level!), then add 6° per edge, 18° across the gap.

b) Resample the signal onto the grid $\theta_m = \theta_0 + m \cdot 0.5°$: for every θ_m interpolate the time $t(\theta_m)$ linearly between the two neighbouring edges, then the signal value $x(t(\theta_m))$ linearly between two time samples. The program compares the grid with the true model angle: report the maximum and rms angle error and explain its size. Rebuild with `-DDT_CAP=20e-6` (edges only known at the sample instants) and compare.

c) The program correlates every engine cycle with the first one, (i) in the angle domain (1440

grid points per cycle) and (ii) in the time domain (segments with the sample count of the first cycle, each starting exactly at its cycle start). Interpret the table and the two plots.

d) The knock-sensor signal contains crank-synchronous parts (combustion vibration $0 \ldots 60°$ after TDC) and fixed-frequency parts (valve impacts: 8 kHz, $\tau = 0.4$ ms). How do the two look in the angle domain during the ramp? Compute the angular length of 3τ of a valve burst at 1500 rpm and 4500 rpm.

e) Which angle step $\Delta\theta$ would be needed to sample the 10.5 kHz knock mode directly in the angle domain at 600 rpm and at 6000 rpm? Which signals should an ECU therefore sample angle-based, which time-based?

Solution

a/b) Key part of the solution:

```
    for (k = 2; k < ne; k++) {
        double d1 = te[k] - te[k - 1], d0 = te[k - 1] - te[k - 2];
        if (kref < 0) {
            if (d1 > 2.0 * d0) {                        /* first reference */
                kref = k;
                the[k] = cam_e[k] ? 0.0 : 360.0;
            }
        } else {
            the[k] = the[k - 1] + (d1 > 2.0 * d0 ? 18.0 : 6.0);
            if (d1 > 2.0 * d0 && fmod(the[k], 360.0) != 0.0)
                printf("warning: gap at unexpected angle %.0f\n", the[k]);
        }
    }
    printf("a) first reference edge: #%d at t = %.3f ms, angle %.0f deg, "
           "last edge angle %.0f deg\n", kref, te[kref] * 1e3, the[kref], the[ne - 1]);

    /* ------------- b) resampling onto the angle grid ----------------- */
    {
        int ke = kref, ks = 0;
        double th0 = the[kref];
        na = 0;
        while (na < MAXA) {
            double th = th0 + na * DTH, tq;
            if (th > the[ne - 1]) break;
            /* angle -> time: linear between the two neighbouring edges */
            while (ke < ne - 2 && the[ke + 1] < th) ke++;
            tq = te[ke] + (te[ke + 1] - te[ke]) * (th - the[ke]) / (the[ke + 1] - the[ke]);
            if (tq > ts[NS - 1]) break;
            /* time -> value: linear between two signal samples */
            tha[na] = th;
            ta[na]  = tq;
            xa[na]  = interp(ts, xs, NS, tq, &ks);
            na++;
        }
    }
```

Output: 2899 crank edges, first reference edge #57 at $t = 38.52$ ms (angle 360°, cam LOW), 35245 grid points. Angle error of the grid: max 0.054°, rms 0.022°. The error is caused by the time stamp resolution: an edge is registered up to Δt_{cap} late, i.e. $\omega\,\Delta t_{\text{cap}} = 27000\,°/\text{s} \cdot 2\,\mu\text{s} = 0.054°$ at 4500 rpm – exactly the maximum found. The linear interpolation between edges (constant speed over 6°) is not the limit: the 1 % second-order speed ripple changes ω by only $\approx 0.2\,\%$ over a tooth. With `-DDT_CAP=20e-6`: max 0.534°, rms 0.222° (and 0.027°/0.011° for 1 µs): the error scales linearly with the capture resolution. This is why ECUs time-stamp crank edges with a hardware capture unit instead of the ADC sample clock.

c) Output (excerpt):

cycle	rpm (mean)	samples/cycle	corr. time domain	corr. angle domain
1	1720	3488	1.000	1.0000
2	1919	3127	−0.030	0.9986
5	2417	2482	−0.131	0.9986
13	3407	1761	−0.035	0.9985
22	4253	1411	0.043	0.9984

In the angle domain every cycle is (up to the 10 mV noise of the model) identical to the first one: correlation 0.998 for all 24 cycles. In the time domain the correlation collapses already in the second cycle, although the segments start exactly at the cycle start and the speed changed by only 12 %: the ion peaks (width a few degrees) at 120°, 300°, … arrive 1 ms to 3 ms earlier and are narrower. The time plot shows the last cycle compressed to 40 % (2.5 cycles in the window of the first one), the angle plot shows two congruent curves. Crank-synchronous signals are *stationary in the angle domain* and non-stationary in the time domain during speed changes; angle-based processing (synchronous averaging, order analysis, Sheet 6) requires the resampling.
d) The combustion vibration is defined by the angle and stays congruent. A valve burst is a resonance with fixed frequency and decay time: in the angle domain its oscillation period is ω/f and changes with speed. $3\tau = 1.2\,\text{ms}$ corresponds to $9000\,^\circ/\text{s} \cdot 1.2\,\text{ms} = 10.8^\circ$ at 1500 rpm and 32.4° at 4500 rpm (one 8 kHz period: 1.1° resp. 3.4°). Fixed-frequency resonances are stationary in the time domain, crank-synchronous ones in the angle domain – no single domain makes everything stationary.
e) The equivalent sampling rate is $f_{\text{eq}} = \omega/\Delta\theta$; $f_{\text{eq}} > 2 \cdot 10.5\,\text{kHz}$ requires $\Delta\theta < \omega/21\,\text{kHz}$: 0.17° at 600 rpm ($\omega = 3600\,^\circ/\text{s}$) and 1.7° at 6000 rpm. A fixed angle step that is fine enough at idle massively oversamples at high speed, a coarse one aliases at idle. Hence: knock sensor (fixed resonances) is sampled time-based with fixed f_s and evaluated in a crank-angle *window*; crank-synchronous quantities (cylinder pressure, ion current, engine speed per tooth, misfire detection) are sampled or resampled angle-based.

Deliverables

4.1: derivations and numbers. 4.2: completed sketch, logic-analyzer screenshot with sampling period and ISR duration, quiet-window results for three noise levels. 4.3: SNR tables (quantisation, jitter, ENOB) with short discussion. 4.4: completed program, angle-error values for two capture resolutions, the correlation table and the two plots with your interpretation.